

Algoritmer og datastrukturer aflevering 1

Group members: *Axel Hauge Kyneb* (AU791500), *Christopher Gascoigne* (AU795734), *Kristian Birkholm Jensen* (AU805541) and *Kristian Estrup Christensen* (AU722500)

Group number: 30

Exercise 1

Our interpretation for this subtask is as follows:

1. Create two vectors called `n` and `m`
2. Let `n` and `m` contain 10 random numbers using the `rand()` function. Store the random numbers in `n` and `m` by using `push_back()`.
3. If `n` and `m` share the same number, the program it is to tell that it has found a duplicate and there will also be a count of the type `int` to keep track of how many duplicates the two vectors share.

a)

Check source code in .zip file

b)

The worst case for this algorithm is if all numbers in both `n` and `m` has a duplicate. If we imagine that is the case, then all lines of code inside

```
for (it = numbersM.begin(); it != numbersM.end(); ++it){...}
```

would be executed in every single iteration of the loop.

Exercise 2

Translating loops to a mathematical equation

$$N(N(N \cdot \sqrt{N})) + N \cdot N$$

reduce

$$N(N^2 \cdot \sqrt{N}) + N^2$$

$$N^3 \cdot \sqrt{N} + N^2$$

using

$$\sqrt{a} = a^{\frac{1}{2}}$$

$$N^3 \cdot N^{\frac{1}{2}} + N^2$$

and

$$a^r \cdot a^s = a^{r+s}$$

$$N^{\frac{7}{2}} + N^2$$

Big-O only cares about the biggest term

$$N^{\frac{7}{2}}$$

so the time complexity of myMethod is

$$N^{\frac{7}{2}}$$

Exercise 3

```
#include <iostream>
#include <chrono>
```

```
// 1
int addUpN(int N)
{
    int sum = 0;
    for (int i = 0; i < N; i++)
    {
        ++sum;
    }
    return sum;
}

// 2
int addUpNSquared(int N)
{
    int sum = 0;
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
        {
            ++sum;
        }
    }

    return sum;
}

// 3
int addUpNCubed(int N)
{
    int sum = 0;
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N * N; j++)
        {
            ++sum;
        }
    }

    return sum;
}

// 4
int addUpNSquared2(int N)
{
    int sum = 0;
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < i; j++)
        {
            ++sum;
        }
    }
}
```

```
    return sum;  
}
```

code)

Check source code in zip file

1a)

Since we are counting from 0 to n, the runtime must increase linearly with increasing size of n: $O(N)$

1b)

Done for $N = 1$ million, 10 million and 100 million

1c)

The running times vary quite a lot from run to run, meaning they are never perfectly linear. The computer handles many other processes simultaneously and the runs will therefore never be the same. However they are relatively linear every time. Example run shown below

```
Time taken (sum = 1000000): 978 microseconds  
Time taken (sum = 10000000): 10330 microseconds  
Time taken (sum = 100000000): 145052 microseconds
```

1d)

As seen below, the optimized flag turned measured time to 0, making it independent of N. We assume that the optimized compiler recognizes the loop's form and that n was given at runtime throwing all of the iterations of the loops away

```
● Axel@Axels-MacBook-Air-6 Week1&2 % g++ -O2 -std=c++17 Task3.cpp -o Task3  
● Axel@Axels-MacBook-Air-6 Week1&2 % ./Task3  
Time taken (sum = 1000000): 0 microseconds  
  
Time taken (sum = 10000000): 0 microseconds  
  
Time taken (sum = 100000000): 0 microseconds
```

2a)

Since we are again counting from 0 to N, but with a nested loop consisting of 2 loops, the Big-O notation must be squared: $O(N^2)$

2b)

Done for $N = 100$, 1000 and 10000

2c)

Again, it varies as stated in previous c task. It does, however, follow quite nicely a squared expansion as n increases as expected. Example run below

```
Algorithm 2:  
Time taken (sum = 10000): 12 microseconds  
  
Time taken (sum = 1000000): 929 microseconds  
  
Time taken (sum = 100000000): 89470 microseconds
```

2d)

Same conclusions as in the first algorithm - it has a closed form for loop that is easy for the compiler to recognize

```
Algorithm 2:  
Time taken (sum = 10000): 0 microseconds  
  
Time taken (sum = 1000000): 0 microseconds  
  
Time taken (sum = 100000000): 0 microseconds
```

3a)

We are again counting up with 2 nested loops, however the inner loop counts from 0 to N^2 while the outer loop counts from 0 to N . This must make the Big-O notation cubed $O(N^3)$

3b)

Done for $N = 10, 100$ and 1000

3c)

Again, it varies as stated in previous c task. It does, however, follow quite nicely a cubed expansion as n increases as expected. Example run below

```
Algorithm 3:  
Time taken (sum = 1000): 1 microseconds  
  
Time taken (sum = 1000000): 900 microseconds  
  
Time taken (sum = 1000000000): 904576 microseconds
```

3d)

Same conclusions as in the first 2 algorithms - it has a closed form for loop that is easy for the compiler to recognize

```
Algorithm 3:  
Time taken (sum = 1000): 0 microseconds  
  
Time taken (sum = 1000000): 0 microseconds  
  
Time taken (sum = 1000000000): 0 microseconds
```

4a)

We are again counting up with 2 nested loops. This is done in a different way than algorithm 2, where each iteration of the inner loop depends on the current value of “i” from the outer loop. It should be done in about half the time (so in reality $\frac{N^2}{2}$) however the same growth rate making the Big-O notation $O(N^2)$

4b)

Done for N = 100, 1000 and 10000

4c)

Again, it varies as stated in previous c task. It does, however, follow quite nicely a squared expansion as N increases as expected. Example run below

```
Algorithm 4:
Time taken (sum = 4950): 6 microseconds

Time taken (sum = 499500): 451 microseconds

Time taken (sum = 49995000): 44366 microseconds
```

4d)

Same conclusions as in the other algorithms - it has a closed form for loop that is easy for the compiler to recognize

```
Algorithm 4:
Time taken (sum = 4950): 0 microseconds

Time taken (sum = 499500): 0 microseconds

Time taken (sum = 49995000): 0 microseconds
```

Exercise 4

Our interpretation for this subtask is as follows:

1. Let a class inherit MaxHeap.h
2. Override the functions and implement them in a .cpp file which in our instance is MaxHeapClass.cpp

a)

Check source code in zip file

b)

bool isEmpty()

The time complexity of this function is $O(1)$. An vector is either empty or it isn't. The number of operations are not influenced by this.

int size()

The time complexity of this function is $O(1)$. Our reasoning behind this is because the number of operations stays the same no matter how large the input is.

```
void insert(const int x)
```

Same reasoning as for `int size()` and `bool isEmpty()`.

```
const int findMax() const
```

The time complexity of this function is $O(n)$. The number of operations are linearly dependent on n . Meaning, as n grows, the number of operations grows linearly with it.

```
int deleteMax()
```

The time complexity of this function is also $O(n)$. Once again, the number of operations are linearly dependent on n .

Exercise 5

a)

Source code uploaded in handin zip.

b)

All books are stored in a single `std::vector<Book>`. This is the simplest ADT that covers what we need: inserting, looping through, and searching for books. `push_back()` is $O(1)$, and since the vector stores elements in contiguous memory, looping through it is fast.

c)

Let n be the number of books currently in the library (`books.size()`).

Function	Time complexity and reasoning
<code>borrowBook(id)</code>	$O(n)$ - Loops through the vector until it finds a matching id. Worst case (not found, or last book) checks every element.
<code>returnBook(id)</code>	$O(n)$ - Same reasoning as <code>borrowBook(id)</code> .
<code>displayAvailableBooks()</code>	$O(n)$ - Has to go through every book to check if it's borrowed and print it if not.
<code>displayBorrowedBooks()</code>	$O(n)$ - Same reasoning as <code>displayAvailableBooks()</code> .
<code>countBooksInCategory(category)</code>	$O(n)$ - No separate counter per category is kept, so it counts by scanning the whole vector every time.
<code>addBook(id, category)</code>	$O(n)$ - Adding the book itself is $O(1)$, but the function also has to look through all the existing books first to check if any category has gotten too big compared to the average. That makes it $O(n)$ overall.
<code>checkBalance()</code>	$O(n)$ - Loops through the vector twice (once for the average before adding, once to check all categories). It calls <code>countBooksInCategory()</code> inside the loop too, but only once per unique category, and since categories are limited to 0–15, that's at most 16 times, so it doesn't change the overall $O(n)$.